

Does an Enforced Semantic Layer Improve Analytical Question Answering? A Preregistered Four-Condition Evaluation on LiveSQLBench Large-v1

Stephanie Jarmak

August 27, 2026

Abstract

A semantic layer rests on the premise that writing business definitions once, and compiling every query through them, makes analytical question answering more accurate than letting a competent agent write SQL against a raw schema. Evaluations of that premise typically change three things at once: whether the business knowledge is available at all, how it is represented, and whether its use is enforced. Changing them together makes attribution difficult. We report a preregistered four-condition evaluation of the Omni semantic layer on LiveSQLBench Large-v1, a benchmark whose 18 PostgreSQL databases (971 tables, 17,749 columns) ship a hierarchical knowledge base of 1,090 business definitions connected by 945 declared dependency edges, with multi-hop resolution required in every database. From the pinned public release we derived 332 eligible **Query** tasks and split them deterministically into 231 development and 101 sealed test questions. Before any sealed generation, label release, or outcome access, the held-out frame was narrowed to the 89 questions on the 16 databases the official Linux loader can populate, and all four conditions were run three times over that identical membership, producing 1,068 generations that were all completed before any output was scored. Under the official-compatible Soft EX scorer, mean one-shot execution accuracy was 10.1% for raw-schema direct SQL, 22.1% for direct SQL with a searchable raw knowledge base, 8.6% for direct SQL with a searchable exported semantic model, and 8.6% for the production-governed semantic-layer product; a corrected multiset comparator gave 10.1%, 19.5%, 8.6%, and 9.7%. The knowledge contrast is the one that moves: searchable business knowledge is worth +12.0 points over the raw-schema baseline (95% interval 5.6 to 18.7), while the governed system is −1.5 points against it (95% interval −7.1 to 4.1) at 3.9 times the median token volume. Mechanism evidence explains the shape of that result rather than resolving it in the product’s favor. Only 17.7% of the 1,090 public definitions compiled into executable objects and 46.9% were deferred across an unresolved grain, so the deployed model published no joins and no measures; every governed development query then took the product’s raw-SQL rewrite path, which means the executed comparison contrasts two agent-authored SQL conditions and does not isolate semantic query composition. We report both frozen scorers without post-result selection, an append-only development record in which one preregistered relationship-path experiment resolved inconclusive under its own coverage rule, a per-attempt telemetry contract in which unobservable quantities are null with a declared reason rather than zero, and the custody boundary that kept hidden gold out of the development workspace throughout. A registered successor condition that deploys the same public knowledge the way the product’s documentation prescribes is specified here with its comparisons and stopping rules fixed in advance; it is confined to the development partition and reports no result in this paper.

Contents

1	Introduction	3
2	Related work	4
3	The benchmark: what the public release contains	6
4	Design, population, and custody	9
5	Conditions, parity, and telemetry	12
6	Two frozen scorers	14
7	Endpoints and analysis plan	15
8	Public-only construction and the development record	16
8.1	What the public knowledge base could be compiled into	16
8.2	Bounding the direct comparator	17
8.3	The development baseline	18
8.4	The control plane, and what it recorded	19
8.5	One deviation, stated in full	20
9	Held-out results	20
9.1	Primary endpoints	20
9.2	The four-condition matrix	21
9.3	Exploratory contrasts	21
9.4	Efficiency and operational reliability	22
9.5	What this result licenses	23
10	Registered successor condition C5, and its pending readout	23
10.1	What C5 changes	23
10.2	Prespecified comparisons	24
10.3	Deployment state at the time of writing	24
10.4	Where the C5 result will go	25
10.5	What C5 will not establish	26
11	What a semantic layer has to represent before it can help	26
12	Threats to validity and limitations	28
13	Standing of the record, and what is still open	29
14	Data and materials availability	30
	References	31

1 Introduction

An analytics team that adopts a semantic layer is making a testable bet: that encoding business definitions once, in governed objects a query engine compiles rather than a language model improvises, makes a downstream agent answer new questions more accurately than the same agent writing SQL against a raw schema. That bet is hard to evaluate from existing comparisons, which typically move several things at once and leave the harness, the question selection, and the number of scored runs unreported.

Three separable things get bundled into such a comparison. *Access*: whether the business knowledge needed to answer the question is available at all. A question about “active high-value accounts” cannot be answered from column names, and no architecture repairs a missing definition. *Representation*: whether that knowledge is a paragraph of prose the model reads or a structured object with declared dependencies, grain, and join paths. *Enforcement*: whether the system may ignore the representation and emit whatever SQL it likes, or must compile its intent through the governed layer and pass its validation. The three carry very different costs, so a comparison that moves all three at once cannot attribute the difference to any of them.

This paper reports an experiment that holds them apart. We evaluate the Omni semantic layer on LiveSQLBench Large-v1, a BIRD-family text-to-SQL benchmark whose 18 PostgreSQL databases ship the material a semantic layer is supposed to encode: a hierarchical knowledge base (HKB) of 1,090 business definitions connected by 945 declared dependency edges, 344 of which point at another derived entry, with multi-hop resolution required in every database. A benchmark in which business knowledge composes is one where the difference between a prose blob and a dependency-aware model can actually show up.

The design is a four-rung ladder, frozen together and evaluated in one sealed event. C1 gives a competent agent the raw schema and lets it write SQL. C2 adds programmatic search over the raw HKB, changing access alone. C3 replaces the raw HKB with a searchable export of the compiled semantic model, changing representation while the agent still writes its own SQL. C4 is the production-governed system. The primary product endpoint is C4 mean one-shot execution accuracy across three independent repetitions of the held-out questions; the primary comparative endpoint is the paired C4 minus C1 difference on the same trials.

What we found. The rung that carries knowledge is the rung that moves. On 89 held-out questions across 16 databases, three repetitions per condition and 1,068 generations completed before anything was scored, official-compatible mean accuracy was 10.1% for C1, 22.1% for C2, 8.6% for C3, and 8.6% for C4. C2 minus C1 is +12.0 percentage points with a 95% interval of 5.6 to 18.7. C4 minus C1 is −1.5 points with an interval of −7.1 to 4.1, and C4 spent 3.9 times C1’s median tokens to get there. The corrected sensitivity scorer preserves every one of those orderings. Making business definitions searchable helped a great deal; compiling them into this deployed model, and then routing queries through the governed product, did not recover that help.

Why the governed rung did not carry it. Two measurements, both taken on public inputs, explain the shape of the result without excusing it. First, the conservative public-only compiler could turn only 193 of the 1,090 HKB definitions (17.7%) into executable objects; 511 (46.9%) were deferred because they crossed a grain the public inputs never resolve. The deployed model therefore published no join paths and no measures. Second, and consequently, Omni’s production agent took the product’s raw-SQL rewrite path on every governed development attempt: all 135 semantic queries carry `rewriteSql: true` with agent-authored SQL and none declares a join path.

The semantic layer still did work, as a field vocabulary the product resolved at rewrite time on 109 of those 135 attempts, but it did not compose the query. The consequence for the study is stated plainly rather than buried in a limitation: C4 minus C3 compares two agent-authored SQL conditions that differ in agent, dialect, accessible surface, and execution contract. It does not isolate enforcement, and no parity fix would make it isolate enforcement, because in neither arm did a semantic layer resolve a join or compile a measure.

Commitments that constrain the result. Several commitments exist because of failure modes documented in the evaluation literature rather than anything specific to this product, and each one is checkable against a preserved artifact. All 1,068 generations completed before any output was scored, so no held-out outcome could reach a prompt, a retrieval configuration, or a retry policy. The development partition was split again into a partition the protocol permits reusing and a metered partition available at most ten times behind a signing boundary; the executed final candidate used neither question-level supervision nor a metered checkpoint, and the metered partition remains unconsumed. Two scorers were frozen before any gold was released, one reproducing the official evaluator’s known lossy behavior and one repairing it, and both are reported without post-result selection. Every attempt carries a telemetry envelope in which an unobserved token count is null with a named reason, never zero. Every condition’s harness is disclosed, because controlled studies place the scaffold’s effect on agent accuracy anywhere from negligible to decisive. The record also carries its own deviations: the held-out frame lost 12 of the original 101 questions to a public-loader defect, one preregistered optimization experiment was scored out of the intended order and resolves as inconclusive, and both are reported in Section 8 and Section 12 rather than described after the fact.

Contributions. A public-artifact reconnaissance of LiveSQLBench Large-v1 including its knowledge-graph structure and the exact behavior of its pinned evaluator; a four-condition ablation separating access, representation, and enforcement, with an executed result and an explicit account of which contrasts survive the measured query path; a custody and freeze protocol under which supervised development on 231 questions could not contaminate the sealed questions, with the freeze commits and artifact digests published; a machine-checkable development control plane in which hypotheses precede changes and rejected experiments cannot be rewritten out of the record; a telemetry contract that keeps correctness out of the immutable generation record and missing measurements distinguishable from measured zeros; a set of product findings, grounded in the same artifacts, about what an import path must represent before a governed runtime can compile anything; and a registered successor condition, C5, that deploys the same public knowledge the way the product’s documentation prescribes, with its comparisons, its stopping rules, and the destination of its still-unproduced numbers fixed in advance in Section 10.

2 Related work

Text-to-SQL benchmarks and the enterprise gap. Cross-domain text-to-SQL evaluation was standardized by Spider [37] and scaled by BIRD, which added dirty real-world values, efficiency measurement, and the observation that external knowledge is often required to bridge a question to a schema [15]. Both established the metric inherited here: execute predicted and gold SQL, compare result sets. The gap to production analytics is documented from several directions. Spider 2.0 moves to enterprise workflows spanning dialects and multi-step transformation and reports that systems strong on earlier suites solve a small fraction of it [14], with a multimodal sibling

covering the surrounding tooling [2]. BEAVER builds questions from an actual warehouse and finds models perform poorly even with prompt engineering and retrieval, attributing the drop to complex schemas, complex business questions, and private data that cannot have been trained on [3]. EntSQL isolates the knowledge dependency: across 1,066 aligned examples in five business domains, 96.0% require information beyond the question and schema, and the best evaluated system reaches 15.9% with long-form documents and 21.4% with concise expert evidence, against a human expert practical ceiling of 84.0% [18]. Recent suites push further into business reasoning and decision framing [17, 30], question whether SQL-string accuracy is the right target [12], extend to database operations agents [4] and autonomous data-modeling agents [34], and propose continuous benchmark generation to keep pace with the systems [28].

The intervention studied here has a direct antecedent. Sequeda, Allemang, and Jacob built an insurance-domain benchmark with an ontology and mappings defining a knowledge graph over a SQL database, and reported zero-shot accuracy of 16% querying the SQL database directly against 54% querying a knowledge-graph representation of the same data [29]. That result motivates the question and also shows what a stronger design must add: a single contrast between “raw database” and “semantic representation” moves access, representation, and enforcement together, and the strength of the direct-SQL arm bounds how much the comparison can establish. LiveSQLBench Large-v1 is used here because it ships the intermediate rung: a structured, dependency-bearing knowledge base from which both a searchable-raw-knowledge and a searchable-semantic-model condition can be constructed from the same public source.

What execution accuracy does not measure. Pourreza and Rafiei show cross-domain text-to-SQL scores are sensitive to evaluation choices, so reported gains can be artifacts of the comparison procedure [27]. SpotIt applies formal verification to the evaluation itself and finds disagreements between result-set equality and query equivalence [11], a question also attacked by asking whether models can decide SQL equivalence [39]. NL2SQL-BUGs targets the complementary error, semantically wrong translations that survive execution-based scoring [19]; ROSE proposes an intent-centered metric [26]; SynSQL synthesizes databases to make execution-based evaluation less sensitive to which rows happen to be present [9]. This is concrete here, because the official LiveSQLBench Query scorer is lossy in ways we reproduced from its pinned source (Section 3). Rather than silently repair or silently inherit the metric, we freeze both a faithful reimplement and a corrected comparator and commit in advance to reporting both.

Contamination and holdout reuse. The benchmark is public, so evaluated models may have seen it. Membership tests for black-box models [25], pretraining-data detection [31], leakage benchmarking [36], open contamination reporting [16], and analyses of what training on a benchmark buys [24] establish that contamination is measurable and common; the practical mitigations are live or continuously regenerated benchmarks [35] and contamination-resistant construction [22]. We treat the dataset card’s contamination-free claim as an official claim, not an established result. The mitigation available to us is different in kind: all four conditions draw on the same public questions and comparisons are paired within question, so prior exposure inflates all rungs together and the contrasts remain interpretable where the absolute level does not. Development itself is adaptive data analysis. Dwork et al. quantify the validity lost under holdout reuse and give mechanisms that preserve it [6]; Hardt analyzes ladder-style adaptive risk estimation [10]; Feldman, Frostig, and Hardt show how outcome structure changes the overfitting rate [7]; Chaibub Neto et al. address the same problem on public leaderboards [23]. Hence the three-way partition of Section 4.

Variance and the harness. Single-run benchmark numbers understate their own uncertainty. Miller sets out error bars for language-model evaluations including clustered and paired analyses [21]; Madaan et al. quantify benchmark score variance across seeds and configurations [20]; Blackwell et al. measure the reproducibility cost of nondeterminism [1]; Coqueret et al. state what randomness obliges researchers to report [5]; Leech et al. catalogue the practices that inflate reported results, including outcome selection after the fact [13]. Separately, any claim about a condition is a claim about a condition inside a harness. Zhang et al. argue as a position that undisclosed harnesses make agent comparisons uninterpretable [38]; that paper reports no effect size, and we attribute none to it. Two controlled studies bound the magnitude and disagree: Vats and Golev measure a 0 to 8 percentage point paired pass-rate difference across three coding harnesses but a 40-fold difference in tokens per solved task [33], while Starace, preregistered, measures an accuracy swing of up to 28 percentage points across three scaffolds on GAIA [32]; Gorinova et al. argue coding benchmarks are misaligned with the agentic systems built on them [8]. A 0 to 28 point range is too wide to assume away in either direction, which is why this protocol neither assumes scaffold effects are negligible nor that they dominate, and why C4 minus C3 is declared a system-level comparison whenever model parity is unavailable.

3 The benchmark: what the public release contains

Everything below was established from pinned public artifacts and pinned public evaluator source. No private attachment, gold SQL, hidden knowledge annotation, or hidden test case was read. “Observed” means reproduced locally; “official claim” means the maintainers state it and this project has not independently established it.

Pinned sources. The dataset is LiveSQLBench Large-v1 at Hugging Face revision `a418e108d5cbb4cf9b783a928eff5e924ad2460d`, with `livesqlbench_large_v1_data.jsonl` verified at SHA-256 `f0e12218cb46f5b6e019908740a0b3303a1f8d1136c661545ad6dd1b4b5444f6`. The evaluator, database build, and reference agent were read at repository commit `e15cd221267e06fabfaf6a3d4a69308280ce9a7c`. The dataset card describes the benchmark as continuously evolving, so the name “Large-v1” does not identify immutable bytes and every experiment record stores both revision and content hash.

Eligibility. The public JSONL holds 480 records with 480 unique `instance_id` values. Filtering on the public `category` field alone yields exactly 332 **Query** records and 148 **Management** records. **Query** is the SELECT-only analytical class; **Management** covers database-management and CRUD tasks, outside the scope of a semantic-layer comparison, and is excluded. All 18 databases contribute, with per-database eligible counts from 10 to 24.

Public records carry `instance_id`, `selected_database`, `query`, `normal_query`, `category`, `high_level`, `conditions`, `preprocess_sql`, and `clean_up_sqls`. The protected fields `sol_sql`, `external_knowledge`, and `test_cases` appear as empty arrays in all 480 records; they are not omitted. The `difficulty_tier` field documented on the card does not exist at this revision, so `difficulty` is used neither for splitting nor for subgroup analysis. Two naming inconsistencies require adapters, not assumptions: the released record uses `clean_up_sqls` while both card and pinned evaluator use `clean_up_sql`. All 332 eligible records have empty `preprocess_sql` and `clean_up_sqls` arrays here, so the mismatch does not change current inputs, but the adapter handles the contract explicitly instead of relying on that emptiness persisting.

Public metadata available for stratification: `high_level` is 181 true and 151 false; `conditions.order` 194 true and 138 false; `conditions.distinct` 17 true and 315 false; and `conditions.decimal` takes values -1 (158), 0 (8), 1 (6), 2 (110), 3 (24), 4 (22), 5 (1), 6 (2), and 8 (1). Database is the dominant stratum. Several joint `conditions` cells are too sparse to cross-stratify without building strata small enough to determine individual questions, so those marginals are audited and reported after allocation rather than used to construct strata.

Scale and schema metadata. The official documentation reports 18 PostgreSQL databases, 971 tables, 17,749 columns, roughly 2,056 rows per table on average, and 746.33 MB total. Counting `CREATE TABLE` statements across pinned public schema files independently reproduces 971 tables, and the column-meaning maps contain exactly 17,749 keys. Two properties shape the transformation work. First, 212 of those values are structured, not plain strings: for JSON and JSONB columns they carry a column-level description plus a `fields_meaning` map describing nested properties, so a transformation that flattens only relational columns discards the definitions that JSON operations need. Second, the schema files are not clean DDL; they interleave `CREATE TABLE` statements with three sample rows per table and retain original quoting, casing, and key declarations, so preparation must parse DDL separately from examples while preserving both as inputs with different provenance.

The knowledge base is a dependency graph. Each database ships a `*_kb.jsonl` whose records carry an integer `id` unique within the database, a `knowledge` concept name, a `description`, a `definition` giving the rule or formula in natural language, a `type`, and `children_knowledge`. Despite the name, `children_knowledge` lists the identifiers the entry *depends on*, with -1 documented as no dependency. Table 4 gives the observed graph statistics; both -1 and the undocumented empty list are normalized to zero outgoing edges.

Table 1: Observed hierarchical knowledge base structure across all 18 databases.

Property	Count
HKB entries (<code>calculation</code> 430, <code>domain</code> 462, <code>value_illustration</code> 198)	1,090
Entries using the documented no-dependency sentinel -1	509
Entries using the undocumented empty list	21
Entries with one or more direct dependencies	560
Direct dependency edges	945
Maximum direct dependencies on one entry	5
Edges whose target itself has dependencies	344
Databases containing multi-hop dependencies	18 of 18
Maximum dependency-chain length	6 edges
Duplicate identifiers / dangling refs / self-edges / cycles	0 / 0 / 0 / 0

This is what makes the benchmark suitable for the question. An HKB entry cannot be treated as an isolated blob of prompt text: 344 edges lead to another derived entry and multi-hop resolution is required in every database. A transformation into a semantic layer must preserve a stable source key, the dependency edges, the original type, the source text, and the compiled objects implementing each entry, and it can be evaluated on whether that composition survives.

Two documentation discrepancies are recorded rather than resolved by inference. The general card describes two HKB formats, structured JSON and an unstructured document, while this snapshot contains only `*_kb.jsonl` and calls itself HKB-JSON. The card also advertises business-rule drift

with versioned definitions, while observed records carry no version or validity field. Whether drift is encoded only in text, deferred to a later release, or present in protected data is unresolved, and will not be inferred from gold.

Official evaluator behavior. For `category == "Query"` the runner discards any customized test-case list and installs one default Soft EX result-equivalence test. At the pinned revision that path removes block and newline-terminated line comments; removes standalone `DISTINCT` while attempting to preserve `DISTINCT ON`; replaces each `ROUND(expression, precision)` with its first argument including nested calls; executes both statements under a 60-second PostgreSQL statement timeout; fetches at most 10,000 rows; normalizes top-level `date` and `datetime` values to `YYYY-MM-DD`, recursively rounds floats and decimals to two places, and serializes lists and dictionaries with sorted keys; requires both normalized results to be non-empty; and compares normalized sequences exactly when `conditions.order` is true and Python sets of rows otherwise.

Consequences that are load-bearing for interpretation follow directly. Unordered comparison discards duplicate multiplicity. Two empty result sets fail rather than match. Date-time normalization discards time of day. The default path hard-codes two decimal places and never consults `conditions.decimal`, and rewrites `DISTINCT` regardless of `conditions.distinct`. The scorer executes the prediction once to detect errors and again inside the comparison, and for a list of statements retains only the last statement’s result. These come from the pinned implementation, not the README description, and any sealed reimplementation must reproduce or explicitly test each.

Reference agent and the knowledge-access boundary. The repository also ships LiveSQLBench-Agent, an orchestrator with eight tools, a 30-step budget, `SELECT`-only exploration, one final SQL submission, and per-task database copies. It is a useful harness skeleton because schema inspection, column meanings, knowledge discovery, execution, and submission are separate operations. It is not usable unchanged: its documented configuration accepts only `lite` or `full` at this revision, and its knowledge endpoint omits `type` and `children_knowledge` from what the agent sees, erasing precisely the dependency structure this study evaluates. It is nonetheless the best public evidence for the intended access boundary. Its tools let a model list database-level knowledge names, fetch a named definition, or fetch all definitions, and its instruction tells the model to discover relevant knowledge; no tool receives hidden per-question `external_knowledge` identifiers as a runtime oracle. This protocol follows the same distinction for C2 through C4.

Provisioning and parity. The official from-scratch path downloads the Large-v1 dumps, builds a PostgreSQL 18 image, imports 18 template databases, and clones from them; the prebuilt compose file declares no Large-v1 service, so the from-source build is the verified official path. The scoring authority is the official local snapshot even if the evaluated product must query a managed mirror. Because a mismatch between generation and scoring copies can manufacture both false gains and false failures, schema fingerprints and deterministic data canaries are compared across copies before any run, and the parity check reports pass, observed disagreement, and could-not-run as three distinct states, so missing access can never be recorded as a passed or failed benchmark question.

4 Design, population, and custody

Estimand. The primary estimand is question-level generalization within a modeled database: given an established semantic model for a database, how accurately does the production-governed system answer questions it has not seen during development, and does the mechanically frozen system generalize to those unseen questions? The held-out unit is the question, not the database. All 18 databases appear in both partitions, and full public schema, column meanings, HKB, and public question text are available during model construction. This matches the product setting in which a team models its data once and users then ask an open-ended stream of new questions. It is explicitly not cold-start performance on an unmodeled database.

A secondary leave-one-database-out study, confined to the development partition, was specified to estimate whether general HKB-to-semantic-layer transformation rules transfer to a domain whose questions were excluded from rule development. It was deferred before execution and is not reported here; the specification is retained because it fixes the design of any later portability claim. Each fold would develop rules from 17 databases, apply them to the held-out database’s public schema, column meanings, and HKB without question-driven evidence from that database, and score that database’s development questions, rotating across all 18, reporting pooled question accuracy and the macro-average across databases with each fold’s question count and modeling effort. If a held-out database receives manual modeling, the result is transformation portability, not zero-shot database generalization. This is a development-set study, not a second test set.

Split. Eligibility uses only the public `category` field: 332 `Query` records in, 148 `Management` records out. Algorithm version 1, under committed seed `omni-livesqlbench-large-v1-split-v1`, allocates test counts across `selected_database` by Hamilton largest remainder, balances public `high_level` within each database, and orders each stratum by SHA-256 of seed, database, `high_level`, and `instance_id`, yielding 231 development and 101 sealed test questions. The 231 development questions are split again under seed `omni-livesqlbench-large-v1-development-split-v1` into 154 dev-A development questions and 77 dev-B checkpoint questions by the same database-first, `high_level`-balanced procedure. Public `conditions` marginals are audited after allocation rather than crossed into sparse strata; the largest post-allocation deviation is `dev-B order`, expected 32 true and 45 false against an actual 37 and 40.

The committed artifacts record source revision, source hash, seed, algorithm version, record identifiers, and subgroup counts. The eligible manifest hashes to `ed8a7b3f55e893e05a9e602b34b39509ca29a30798afd63fd9a521f5ffbc7d0e`, the train identifiers to `ecdd1e4cd425e41970d3d4e96d7c3cb044cb78d94e34621d320e3d542a5d1c46`, and the test identifiers to `c7d4eb1aff3e9be9ffa015adcae96bb9294adf138870db24cdfbed811a9de76f`. Tests establish that identifiers are unique, train and test are disjoint, their union is the 332-record eligible manifest, counts are 231 and 101, all 18 databases appear in both partitions, regeneration from the pinned seeds and source hash is byte-identical, and no protected field has entered a committed artifact. Manifests are regenerated, never hand-edited. Membership never changes in response to any observed outcome.

The executed held-out frame. The official Large-v1 Linux loader does not populate two databases. It skips 34 declared tables in `mental_health_large` and 37 in `organ_transplant_large`, because the capitalization of those archive filenames differs from the names the loader requests, so the benchmark’s own reference SQL cannot execute there. Twelve of the 101 sealed questions belong to those two databases. Before any sealed generation, any label

release, and any outcome access, the held-out frame was amended to the 89 questions on the 16 databases with verified governed deployments, and all four conditions and all three repetitions used that identical membership, for 1,068 scheduled attempts. The amendment used no question content, no gold, no annotation, and no correctness; it used only which databases the public loader can populate. It narrows the held-out estimand from all 18 benchmark databases to those 16 deployable databases, which is a scope limitation rather than a set of missing outcomes, and it is reported as such in Section 12. The same loader defect removes 18 of the 154 development questions from official scoreability; the development schedule still lists all 154 and reports those 18 as scorer-conformance exclusions, leaving 136 answerable development questions for the governed arm.

Two further database exclusions apply only to the public direct-SQL development baseline and not to the held-out frame. `archeology_scan_large` repeatedly failed to return a usable direct answer across distinct retrieval settings, and `cybermarket_pattern_large` was excluded after its first immutable launch failed read-only privilege attestation, even though the external credential was later repaired. Both were fixed in a committed exclusion artifact before the affected generations, and their records stay outside the wrong-answer and missing-row categories.

Information tiers. Table 2 states what may be used where. The governing rule is that hidden training annotations could influence a reusable system under the protocol but may never become question-specific runtime input. The executed final candidate does not use that supervision. Development `external_knowledge` identifiers are privileged diagnostic-oracle metadata: they may be consulted *after* a prediction, to test whether the required public HKB nodes were represented, whether dependencies compiled, and whether the system could reach them. They may not select runtime context. Question-identifier rules, hidden-identifier retrieval hints, and question-specific gold-derived transformations are prohibited; semantic changes must be reusable definitions, database-level modeling, transformation rules, or general harness behavior. If sample queries are used, each must encode a reusable pattern and not a question-to-gold pair, and material use requires an ablation.

Table 2: Information tiers.

Information	Development use	Runtime use	Held-out
Public schema, HKB, column meanings, questions	Allowed	Allowed	Frozen
Hidden train SQL, test cases, knowledge identifiers	Offline supervision only	Prohibited	Frozen
Hidden test SQL, test cases, knowledge identifiers	Prohibited	Prohibited	Sealed evaluator only

Gold custody. The complete private attachment stays under human custody outside the repository and outside any agent-accessible workspace; its SHA-256 is computed without displaying or parsing its contents. Only after the split commit exists may a custodian-run extraction emit exactly the 231 training records into an ignored directory. The release tool verifies the canonical train identifiers and split metadata against the externally recorded pre-gold commit rather than the mutable current branch, rejects sources inside the workspace, refuses overwrites, writes mode 0600, and reports only counts and hashes. It never releases the 101 held-out records. Logs may contain identifiers,

hashes, counts, status codes, and aggregate scores, never hidden SQL, hidden knowledge identifiers, or test-case bodies.

Checkpoint correctness on **dev-B** sits behind a separate guardian boundary. The guardian signs exact receipt bytes with a private key held outside agent scope; the checkpoint capability verifies the detached signature against a separately recorded public-key digest that is part of the protocol-frozen configuration and cannot be supplied by a caller, rejects replayed receipt or output hashes, and irreversibly increments a counter. Each consumption writes both an immutable checkpoint manifest and a numbered allocation marker, and both must agree byte-for-byte and be present in the current commit before another checkpoint is permitted, so marker deletion, manifest deletion, receipt replay, and local counter reset all fail closed. The protocol maximum is ten **dev-B** evaluations, with five to ten intended over the project.

Baseline ordering and the two freezes. Before hidden training annotations are released, a public-only baseline is preserved: transform public schema, column meanings, and HKB into the semantic layer deterministically; validate the model structurally; freeze the mechanical transformation revision and model artifacts; generate baseline outputs on all 231 development questions; and only then release train annotations for scoring and failure analysis. This separates initial transformation quality from any improvement that might have been obtained through supervision. The baseline may be scored once its immutable outputs exist, but may never be regenerated retrospectively. The protocol permitted supervision on development questions, but the executed final system is the frozen mechanical baseline and receives no question-level supervision. It is still not a cold-start database evaluation because public schema, column meanings, and HKB are compiled into the model.

Freeze A, the pre-gold protocol freeze, commits the eligible population, both splits, custody and information rules, the C1 through C4 definitions, both scoring policies, repetition, order, and failure rules, endpoints and statistics, unblinding and adjudication policy, the ledger schema, and the guardian public-key pin. Its commit hash is recorded before any hidden development label is released, and the control plane verifies configuration and manifests against that recorded commit, not the working tree. Freeze A deliberately does not freeze the final system. *Freeze B*, the pre-test system freeze, occurs after the recorded decision to cut optimization and consume no checkpoints, and before any test generation: it commits and hashes the semantic models and transformation code, all four harness configurations and prompts, retrieval configuration and allowed tools, model and provider information and budgets, retry and failure-classification policy, database snapshot identifiers, the trial schedule, and the evaluator version. All 1,068 scheduled generations then complete, and only then are they scored. No held-out result may feed a model, harness, prompt, retrieval, or retry change.

Both freezes executed. The untuned system that produced the held-out generations is frozen at successor commit `8b0c7393d564d9ecc2c2f84ba7446d610c1a0a6d` with direct-child control commit `94cc0d9483c944d7dc13ed651c8fc2ef077f33ab` and Freeze-B manifest SHA-256 `e1c9f1967422822c848c18a17ba759d4e4fbc7f21aa0fe3ae1045b9236ae4730`. Scoring ran from correction-forward system `0a5aee423b4a0d5bb396b3d9764f8e9e24f31254` against control `fe4660df8dacdca07da310ddfa4158b82895ba9` under Freeze-B v10 SHA-256 `ff10083bf70d82bd483d12e98751d9bf7f5d4236c42fac3ba921405d87953a05`, preserving the generation binding above. The correctness-free scoring receipt hashes to `534e28b954d4d13dfdd9100fc6a184fba3eb3720d8cd7cf7d43c92713cb258f7`. No optimized held-out candidate was constructed after the sealed result.

5 Conditions, parity, and telemetry

The four rungs. All four conditions are reasonably developed on training data, frozen together, and evaluated in the same sealed event. Table 3 states what each changes.

Table 3: The four frozen conditions as specified. The enforcement column states the design intent; what the governed condition did at run time is measured later in this section.

Condition	Knowledge	Representation	Enforcement
C1 Raw SQL	Public schema	Raw schema	None
C2 HKB reference	Public schema and HKB	Searchable raw HKB	Optional
C3 Model reference	Public schema and HKB	Searchable exported semantic model	Optional
C4 Governed	Public schema and HKB	Semantic model	Production harness enforces semantic compilation and validation

C2 and C3 receive comparable programmatic search rather than being made to ingest a roughly 84K-token export in one prompt; the difference between them is the representation, not the retrieval budget. C1 through C3 share one pinned direct-SQL model and harness and match question wording, database, execution access, budgets, retries, and scorer. C2 is a substantive condition and not a bridge: business semantics supplied as searchable context may account for a material part of the total C4 minus C1 gap, and evidence about semantic context informs C2 minus C1 only.

C4 uses the production-default configuration, because production fidelity outranks forcing model parity for the primary product endpoint. The exact C4 models could not be observed and invoked independently: model version, retry count, and validation-attempt count are unavailable for every held-out C4 attempt. Provider, observable identifiers, and settings are recorded where the product exposes them, and the interpretation is therefore the conservative one stated in advance. C4 minus C3 is a production-system comparison and is not the causal effect of enforcement. All conditions receive real development rather than leaving C1 through C3 as straw comparators; per condition we record prompt revisions, training evaluations, major interventions, approximate effort, and the actor or automation that performed the work. Exact equal person-hours are not required.

Parity policy. Pre-treatment resources are matched where matching is meaningful: database snapshot, credentials and permissions, public information, task specification, model identity when possible, reasoning and token ceilings, and retry ceilings. Post-treatment behavior is not artificially equalized. Actual tool calls, tokens consumed, latency, database-query count, retries, and cost are measured as outcomes, because a governed system that spends more to be right and a direct agent that spends less to be wrong differ in a way an equalized budget would hide.

Telemetry contract. Every attempt produces an immutable generation envelope containing no correctness information. It records run and attempt identifiers, condition, repetition, partition, and timestamps; provider, model, and version; token counts and their source; cost and its source; wall-clock latency, tool calls, database queries, retries, and validation attempts; the generation outcome (**answered**, **refused**, or **errored**), terminal failure origin and class, the generated SQL

or governed query, semantic objects, and fixed compiler, validation, and execution statuses; and a relative path, schema version, and SHA-256 for the ordered raw trace, or an explicit degraded reason when trace capture is unavailable.

Unobserved counts are null and must be named in an explicit unavailability list; they are never encoded as zero. Token and cost values separately declare whether they are provider-reported, derived, or unavailable. Complete traces must reconcile token, tool-call, database-query, retry, and validation totals to the attempt envelope, and truncated traces carry an explicit degraded reason. Scoring produces a *separate* record carrying `correct`, `wrong_answer`, or `refused_or_error`, hash-bound to both the generation file and every attempt record; it cannot mutate or copy the prompt, query, result, or telemetry. Raw traces and generated SQL live only under ignored run roots, and committed artifacts contain hashes and permitted summaries.

Two consequences are worth stating explicitly. The three-state attempt outcome is distinct from the three-state generation outcome, so a confident wrong answer and a safe refusal are never collapsed even though both miss the benchmark. And for composite production workflows, an aggregate model label must not be presented as exact model parity when internal routing is opaque; where the product exposes per-stage model and usage, raw capture preserves them.

Capture verification gate. Before any 231-question baseline or expensive experiment, each implemented condition had to produce one public unscored smoke attempt whose envelope validates, whose trace hashes and redaction verify (or whose unexposed fields carry an explicit degraded reason), whose system database-query counts exclude scorer and gold execution, and whose evaluated-system retries are distinguishable from benchmark-infrastructure reruns. Coverage was recorded per field and condition. The gate passed for all four conditions, and the field-level coverage it recorded is what makes the unavailability statements in Section 9 auditable rather than rhetorical: latency coverage is complete in every condition, token, tool-call, and database-query coverage is 267 of 267 attempts for C1 through C3 and 265 of 267 for C4, and provider-reported dollar cost is 267 of 267 for C1 through C3 and 0 of 267 for C4.

What the governed condition turned out to be. C4 was preregistered as the condition in which the production harness enforces semantic compilation and validation. Measured on the frozen development baseline, it is also an agent authoring SQL. All 135 captured governed semantic queries carry `rewriteSql: true` and `aiGenerated: true` with hand-authored SQL in `userEditedSQL`; `generated_sql` is null on every attempt; zero queries declare a join path; and `join_via_map` is empty throughout. The choice of path was the product’s own. The harness posts only a model identifier, the bare question, and a branch identifier, and exposes no mode flag; a repository-wide search establishes that no harness code ever writes `rewriteSql`.

The reason the path existed is in the deployed model rather than in the agent. Because the conservative public-only compiler deferred 46.9% of HKB definitions across an unresolved grain (Section 8), the deployed topics publish no joins and no measures, so for any cross-table or aggregate question no compiled path was available to take. The semantic layer still did work as a field vocabulary that the product resolved at rewrite time: 109 of 135 attempts reference at least one compiled dimension, and 39 expand an HKB-backed derived definition into the executed query. It did not compose the query. Two consequences follow and are carried through the rest of the paper. C4 minus C3 does not isolate semantic-layer query composition, and no achievable model parity would make it do so, because neither arm resolves a join or compiles a measure. And C4’s structural aggregates are computed from `userEditedSQL`, so they describe agent-written queries in all four conditions. The full measurement, including every prior claim it retracts, is in the query-path

disclosure listed in Section 14.

6 Two frozen scorers

Both comparators are frozen before hidden development labels are released, both consume already-executed rows, and neither reads benchmark records, question identifiers, gold SQL, or private annotations. Both are reported. Neither is selected after results are seen.

Official-compatible Soft EX. Version `livesqlbench-soft-ex-e15cd221-v1` reproduces the Query comparison path of the public evaluator at commit `e15cd221267e06fabfaf6a3d4a69308280ce9a7c`, including the SQL rewrites, result normalization, empty-result rule, and ordered-versus-set comparison described in Section 3. The reproduction is deliberate and extends to behavior we regard as lossy: a line comment at end of input without a trailing newline is retained, because that is what the pinned code does. This is the benchmark-comparability score, and it will not be quietly improved.

Corrected sensitivity scorer. Version `omni-multiset-decimal-v1` leaves authored SQL unchanged and compares results as stable typed values. Unordered results are compared as multisets, preserving duplicate rows; ordered results as sequences; two empty results match while one empty result does not; null, boolean, numeric, string, date, mapping, and sequence values retain distinct canonical representations; dates and datetimes normalize to `YYYY-MM-DD`; dictionary key order does not affect equality. Public `conditions.decimal` values receive frozen meanings: `-1` preserves the canonical numeric value without rounding, and a nonnegative value rounds decimals, floats, and integers to that many places using decimal half-up rounding. Unknown values, booleans in place of integers, a missing `decimal`, and non-boolean `order` fail validation, so a future change in the public data becomes explicit rather than silently changing scoring.

Pure result comparison and SQL execution and rewrite behavior are versioned separately and passed public conformance fixtures before Freeze B. The sealed evaluator’s integration layer executes rewritten candidate and gold SQL against the same isolated PostgreSQL state for the official score and unchanged authored SQL against equivalent state for the sensitivity score, applies the public preprocessing and cleanup lifecycle, preserves multi-statement ordering with stable driver types, enforces the preregistered timeout and retry rules, distinguishes candidate-system failures from benchmark-infrastructure failures, records the three-state attempt outcome separately from result equality, and keeps hidden test SQL and rows inside the sealed boundary. Both policies were published for all four conditions in one atomic scoring run, and neither was selected after results were seen; Section 9 reports both throughout. Where the two disagree the disagreement is small and never reverses a rung ordering: the largest gap on the held-out frame is 2.6 points, on C2.

Post-hoc analyses. After final scoring, a sealed audit normalizes development and test gold SQL into structural templates, reports overlap prevalence, and, if sample size permits, reports test accuracy on non-overlapping templates. That audit and the adjudication path below remain specified and unexecuted; both sit behind the sealed-label custody release, which is a human-controlled action. Suspicious or condition-discordant items may enter a preregistered post-freeze gold-quality adjudication with condition identity blinded where practical. Neither audit can change the split, the primary result, or the development system, the original official score remains intact, and any adjudicated result is labeled a separate sensitivity analysis rather than replacing benchmark truth.

7 Endpoints and analysis plan

Primary endpoints. The product-performance endpoint is C4 mean one-shot execution accuracy across three sealed repetitions, estimated from 267 question-repetition attempts with question-clustered uncertainty. C4 repetition-one accuracy is reported separately as the intuitive single-invocation view. There is no majority vote, best-of-three, or selective-retry score. The comparative endpoint is the paired C4 minus C1 execution-accuracy difference on the same trials, paired within question and repetition and clustered by question. It is a system-level contrast between two reasonably developed systems.

Exploratory contrasts. C2 minus C1, C3 minus C2, and C4 minus C3 are prespecified exploratory mechanistic evidence, reported with effect sizes, transition counts, and uncertainty. Modest rung differences will not be promoted on significance. Where C3 and C4 lack model or system parity, C4 minus C3 is explicitly system-level. Repetition-one McNemar analyses may appear as sensitivity checks; naive McNemar is not applied to 267 correlated observations. With 89 questions, small paired deltas are imprecise, and effect sizes and failure transitions take precedence over p-values. Section 9 reports every contrast in this paragraph, under both scorers, without post-result selection between them.

Repetitions and reliability. Each condition runs three independent repetitions on every test question, for 1,068 frozen outputs generated before any scoring. Trial order is a committed deterministic block-interleaved permutation over question, condition, and repetition, with repetitions of one question separated, and each trial records timestamps and observable model identifiers. The reliability analysis reports per-repetition and mean accuracy; per-question 3/3, 2/3, 1/3, and 0/3 correctness; correctness-flip rate separately from result-set changes; refusal and error rate and its stability; latency and cost distributions; and question-clustered bootstrap intervals that resample questions as clusters retaining all condition and repetition outcomes. Correctness stability and answer stability are reported separately: the same wrong answer three times is stable and incorrect, while different SQL returning the same correct result is outcome-reliable. SQL-text variation is diagnostic only.

Co-outcomes. Alongside accuracy we report confidently-wrong rate, refusal and error rate, tokens per correct answer, median and interquartile range for tokens and latency, tool calls and database queries per attempt and per correct answer, retry and validation distributions, telemetry coverage, and terminal failure vectors by condition. Operational numerators include every valid attempt; comparative views use matched question-condition-repetition populations. Cost comparisons require equivalent output accounting, since a cheaper condition that fails to answer is not more efficient. Each major intervention reports whether it increased correctness, reduced confident errors, converted wrong answers into refusals, or only shifted the failure mode, so a flat accuracy result can still be product-relevant.

Subgroups. Prespecified descriptive subgroups are database, public `high_level`, and public `conditions` values with adequate cell size. Rare cells and per-database test estimates are descriptive and will not be mined for significance. Relative improvement is reported only where the baseline denominator is nonzero.

Failure taxonomy. Failure categories are built from training evidence rather than forced into a prewritten list, and each records count, denominator, representative non-private examples, detection evidence, and a primary ownership hypothesis (model, semantic model, harness, product, environment, evaluator, or benchmark) that changes only with evidence. For a failed development question whose offline annotation references HKB knowledge, we classify the earliest supported failure point on a mechanism ladder: knowledge absent from the semantic model; knowledge present but the dependency graph wrong; knowledge correct but not retrieved; retrieved but misinterpreted; representation correct but compilation failed; compiled query correct but validation or harness behavior changed the outcome; or model reasoning failed despite a correct available representation. Hidden knowledge identifiers stay in offline diagnosis, and the published taxonomy records aggregate classifications only.

Prioritization uses the qualitative product of prevalence, plausible fixability, and generality, with a model or reviewer supplying the semantic judgment and its evidence; application code validates record structure and enforces custody policy rather than embedding keyword heuristics for failure meaning.

Retry and rerun policy. The evaluated system’s normal production retry policy is part of the frozen system. Exhausting it, returning no answer, generating invalid SQL, or timing out inside the agent or harness is an incorrect system outcome. A trial may be rerun only when failure is demonstrably outside the evaluated system: sealed-evaluator process failure, benchmark database unavailability, or transport failure before the evaluated service accepted the request. A rerun keeps the same trial identity and appends a reason with both attempt records. Rate limits, 5xx responses, and timeouts are classified in advance by which system owns the retry boundary, and are never reclassified after scoring. No trial is ever rerun because its answer was wrong.

8 Public-only construction and the development record

The final candidate is the frozen mechanical baseline. It received no question-level supervision, and the metered **dev-B** partition was never consumed. This section reports how that candidate was built from public inputs only, what the development partition showed about it, and what the append-only control plane recorded about the interventions that were proposed against it.

8.1 What the public knowledge base could be compiled into

Before any hidden development label was released, public schema, public column meanings, and the public HKB were transformed deterministically into semantic artifacts. Every knowledge node received exactly one disposition: **compile**, a defensible same-table, same-grain scalar definition; **context_only**, useful semantic text with one exact field target but no safe executable object; **defer_cross_grain**, a definition requiring unresolved identity, relationship, cardinality, aggregation, temporal, or ordering semantics; or **unsupported**, missing, ambiguous, or currently non-representable inputs. Guessing a join or an aggregation would have raised apparent model coverage while destroying the governance claim the benchmark is meant to test, so the rule is deliberately conservative.

Table 4 is the single most consequential measurement in the study, because it determines what the governed condition could possibly have done. Fewer than one definition in five became an executable object, and almost half were deferred for crossing a grain the public inputs never resolve. Within the 17-database fan-out beyond the canary, the three most frequent recorded loss codes were **cardinality_unknown** (398), **aggregation_unspecified** (314), and **cross_grain_no_identity**

Table 4: HKB translation coverage across all 18 databases.

Disposition	Definitions	Share
Compiled	193	17.7%
Context only	193	17.7%
Deferred cross-grain	511	46.9%
Unsupported	193	17.7%
Total	1,090	100.0%

(308). Domains built from row-local physical or sensor definitions, such as planets and solar panels, compiled comparatively well; residential and reverse-logistics models retained useful context but compiled no HKB definition safely under the same rules.

This measures transformation coverage and not question correctness. A deferred definition may never be needed for any benchmark question, and a compiled definition may still be retrieved or interpreted wrongly. What it does establish is that the deployed model published no join path and no measure, which is the precondition for the query-path finding in Section 5.

8.2 Bounding the direct comparator

The first end-to-end vertical slice ran one public development question before anything scaled. The first C1 attempt passed the public-question, database-parity, read-only, model-identity, and first-turn gates, and then its opening schema inspection returned all 51 tables of the database and the next model turn exceeded its budget before producing any SQL. Whole-database inspection was replaced with deterministic query-directed search over the same committed public schema: the model must supply a non-empty query, and the tool returns at most four matching tables inside a 64 KiB payload. C1 through C3 share that retrieval contract.

Table 5: One question, three integration attempts. Each row is a single attempt on one database, not a population estimate.

Stage	Terminal state	Tokens	Cost	Latency
Whole-schema C1	Budget error	173,365	\$1.74	26.0 s
First bounded diagnostic	Public-ID validation error	1,585	\$0.02	3.0 s
Reviewed bounded C1 canary	Answered	33,445	\$0.21	40.9 s

The intermediate failure in Table 5 is preserved on purpose. The first bounded attempt still returned no answer, because a generic secret heuristic rejected canonical public foreign-key identifiers; a typed, fail-closed validation rule fixed that boundary while keeping the credential checks. Keeping the failed row prevents the whole change from being attributed to retrieval alone. The four-match window itself remains unvalidated as a choice: a later attempt on a different archeology question reached five four-table searches, cost \$7.49, and ended in a model budget error, and reducing the window to two tables cut that attempt to four searches and \$4.32 but still returned no answer. The cheaper failure was no evidence for the smaller window, so the change was reverted, and a 20-question sensitivity arm over the match cap was frozen and then deferred before execution.

The product lesson generalizes past this comparator. Tool payload bounds are part of agent quality: a semantically valid tool call can make a system unusable by consuming the remaining

inference budget, and a safety filter that treats every identifier as untrusted free text can reject legitimate public metadata.

8.3 The development baseline

The 630-attempt direct baseline was frozen by content hash before the train-only gold release. Its exact development intersection contains 420 attempts over 140 of the 154 development questions; 14 questions have no baseline output and were left missing rather than backfilled. Gold conformance left 122 questions per condition scoreable under the official scorer, and 121 under the sensitivity scorer, which retained one fewer result that exceeded its fixed normalization limit.

Table 6: Direct-SQL development baseline, exploratory. Percentages are over each condition’s scoreable denominator.

Condition	Correct	Wrong	Refused/error	Official	Sensitivity
C1 raw schema	9	80	33	7.4%	7.4%
C2 searchable raw HKB	29	91	2	23.8%	23.1%
C3 searchable exported model	16	74	32	13.1%	11.6%
All direct conditions	54	245	67	14.8%	14.0%

Two things in Table 6 survived into the held-out result. The ordering did: searchable raw business knowledge produced the strongest direct baseline, and converting it into the bounded structured model gave back more than half of that gain. And wrong answers dominated: 245 of 366 scoreable attempts returned a wrong result rather than declining to answer. C2 also had the highest capture-level completion rate on the 16-database frame at 97.9%, against 75.9% for C1 and 74.8% for C3, but 91 of its 122 scoreable attempts were wrong, so completion cannot stand in for correctness.

The governed arm was budgeted and scheduled separately, over the full fixed development schedule rather than the direct arm’s represented frame. Of its 154 scheduled questions, 136 were answerable and all 136 executed: 9 correct, 93 wrong, and 34 refused or ended in a system-contract error, for 9/136 (6.6%) official accuracy; the sensitivity scorer retained 135 scoreable questions with the same 9 correct and 93 wrong, for 9/135 (6.7%). An aggregate-only alignment to the 122 questions scoreable in all four conditions gives the governed arm 5 correct, 83 wrong, and 34 refused or system-error, or 5/122 (4.1%), against 7.4%, 23.8%, and 13.1% for C1 through C3 on that same frame. The paired C1/C4 table on those 122 questions holds 3 correct in both, 2 correct only in C4, 6 correct only in C1, and 111 correct in neither. The governed arm’s low result is therefore not an artifact of its wider denominator. No question identity, SQL, row value, annotation, or per-question label left custody during that alignment.

A structural diagnostic over the SQL text, carrying no question identity, no SQL body, no result value, and no hidden annotation, found that wrong answers used more relations on average in every direct condition: 3.16 against 2.00 in C1, 3.28 against 2.48 in C2, and 3.04 against 1.81 in C3. Thirty of 31 window-query attempts and 25 of 28 DISTINCT-query attempts were wrong, while the mere presence of a join or an aggregate separated correct from wrong hardly at all. The governed arm ran in the same direction: parseable correct queries averaged 1.67 relations against 2.62 for wrong answers and 2.88 for system errors. These are associations. They point at relationship, grain, and dependency handling as the next mechanism family to test, and they establish nothing causal.

8.4 The control plane, and what it recorded

Optimization on the development partition is the part of an evaluation most likely to leak, to overfit, or to be narrated favorably afterward, so it was constrained by a control plane whose records are append-only and whose failed experiments therefore remain in the record. Routine optimization received a derived public view containing only the 154 development identifiers and never a checkpoint or held-out outcome. Generation artifacts may carry generated SQL or a governed query, latency, token and cost observations, non-secret tool failures, and the semantic-object identifiers the system used; they may not carry gold SQL, test cases, hidden `external_knowledge` identifiers, expected results, or scores, and the validator rejects protected keys recursively, so nesting a hidden field does not bypass the boundary. Correctness arrives only in a separate immutable score artifact whose path and expected digest are mandatory for every scored decision, and the control plane stores component hashes plus a combined candidate hash so a guardian receipt cannot be rebound to a different generation or score artifact. Each iteration appends a proposal *before* the system changes, stating the affected failure class, the mechanism, the predicted direction, the regression risk, the surface, and why the intervention is reusable; it then appends exactly one of `KEEP`, `REVERT`, `INCONCLUSIVE`, `INVESTIGATE`, or `ARCHIVE`. Every kept intervention declares its scope, question-specific changes are prohibited outright, and benchmark-specific candidates may be archived as evidence but never promoted into the final system. The objective is ordered rather than scalar, so an accuracy increase alone cannot authorize acceptance.

Four intervention families were preregistered against the mechanism ladder: same-grain dependency composition (E01), FK-backed grain relationships (E02), bounded semantic descriptions (E03), and a broad HKB-context negative control (E04). A fifth, E05, was registered later against the governed arm’s `UNKNOWN`-type result-contract failures. None was promoted.

E01 resolved inconclusive on inspection: the frozen baseline already carried 48 dependency-bearing elements, 70 executable dependency edges, and depth three, so there was no contrast left to run.

E02 is the substantive one. It declares FK-backed relationships, which is exactly the ingredient whose absence left the rewrite path as the only route to cross-table access, so it is a direct test of the mechanism rather than a candidate picked from a structural correlation. Of 1,228 public foreign keys, 1,049 pass the conservative cardinality contract, and the bounded artifact emits 91 relationships across 16 databases and 67 source topics with zero metric-disposition changes. Its deployment verified all 16 targets with exact readback, and its single 136-attempt development generation completed and was frozen. It then failed its own coverage gate: 19 of the 136 attempts captured no scoreable outcome. Fourteen were saved semantic queries rejected by the result-type capture contract, and five were genuine transport failures that retained no semantic query at all, so the preregistered complete-136 score could not be produced without a new model attempt, which the no-rerun rule forbids. E02 is recorded `INCONCLUSIVE`.

Because the answer to a stopped experiment is not a rerun, a no-rerun diagnostic was applied offline to the 117 captured answers under both frozen scorers, with no model answer regenerated. Official accuracy on that subset was 11/117 (9.4%) for E02 against 9/117 (7.7%) for the frozen governed baseline on the identical coordinates, a paired difference of +1.7 points, with four transitions to correct and two regressions from correct; sensitivity gave 10/116 (8.6%) against 9/116 (7.8%), or +0.9 points. The full-frame official bounds run from 11/136 (8.1%) if every missing outcome is wrong to 30/136 (22.1%) if every one is correct, and treating the 14 contract failures as failures and only the five transport losses as unresolved still leaves an upper bound of 16/136 (11.8%). Bounds that wide, over that few transitions, support further study and not a claim that E02 improved accuracy.

E05 was stopped by its own precondition before it consumed a live attempt. It proposed declaring explicit output types on compiled semantic fields to address the 31 UNKNOWN-type contract failures, and required at least 16 of those 31 attempts to have selected a compiled derived field. Measured offline on the immutable generation records, the ceiling is 6 of 31, and 24 of the 31 select no compiled bundle field of any kind, so no declaration on a compiled field could reach them. E03 and E04 were never run and fell out of scope.

8.5 One deviation, stated in full

The untuned held-out arm was scored before E02’s development execution finished, which reverses the ordering that the later lean optimization extension specified. E02 had already been selected and preregistered as a relationship-path contrast, and its general compiler change predates the held-out results, so the frozen untuned comparison is unaffected: every held-out generation completed before scoring, and both scorers were published in one atomic run. The cost falls entirely on optimization. Because held-out aggregates are now visible, they may not drive a new intervention edit, a checkpoint consumption, a promotion decision, or an optimized held-out arm, and E02 cannot be rerun or promoted into a sealed successor. The study therefore reports a valid frozen four-condition held-out comparison and a separate pre-specified development mechanism experiment. It makes no claim of held-out improvement from tuning. One later correction generalized the publication of already-normalized relationship endpoints in response to public validator evidence; it used no held-out outcome and changed no experimental choice.

9 Held-out results

The frozen comparison covers 89 held-out questions and 1,068 scheduled attempts: four conditions, three repetitions, identical membership. Every generation completed before any output was scored, and the scorer published both preregistered policies in one atomic run. Everything below is derived from identity-free aggregates. No question identity, SQL text, result row, hidden annotation, or per-question correctness left custody to produce it.

9.1 Primary endpoints

Table 7: Preregistered primary endpoints under both frozen scorers.

Scorer	Endpoint	Estimate	95% interval
Official Soft EX	C4 mean one-shot accuracy	8.6%	3.7–14.6%
Official Soft EX	C4 repetition-one accuracy	7.9%	3.4–13.5%
Official Soft EX	C4 minus C1, paired	−1.5%	−7.1–4.1%
Corrected multiset	C4 mean one-shot accuracy	9.7%	4.5–15.7%
Corrected multiset	C4 repetition-one accuracy	9.0%	3.4–15.7%
Corrected multiset	C4 minus C1, paired	−0.4%	−6.4–5.6%

The production-governed system answered roughly one held-out question in twelve, and its paired difference against a competent agent writing SQL against the raw schema is negative under the official scorer and negative under the corrected one, with intervals that comfortably include zero in both directions. The primary comparative endpoint gives no evidence that the governed path

outperformed the direct baseline on these questions, and the interval is wide enough that it also gives no evidence of a large loss.

9.2 The four-condition matrix

Table 8: All four conditions under both scorers. Refused/error is the scorer’s third outcome over the 267 scoreable attempts per condition. Error rate is narrower: the share of all 267 scheduled generations whose raw terminal outcome was **errored**. Pass³ is the share of the 89 questions correct in all three repetitions, and flips counts questions correct in only one or two. These columns measure different properties and do not sum.

Scorer	Cond.	Accuracy	Wrong	Ref./err.	Error rate	Pass ³	Flips
Official Soft EX	C1	10.1%	56.2%	33.7%	18.7%	6.7%	6
Official Soft EX	C2	22.1%	50.6%	27.3%	19.9%	15.7%	12
Official Soft EX	C3	8.6%	53.2%	38.2%	16.1%	4.5%	6
Official Soft EX	C4	8.6%	77.2%	14.2%	14.2%	6.7%	4
Corrected multiset	C1	10.1%	56.9%	33.0%	18.7%	6.7%	6
Corrected multiset	C2	19.5%	54.7%	25.8%	19.9%	14.6%	10
Corrected multiset	C3	8.6%	54.7%	36.7%	16.1%	4.5%	6
Corrected multiset	C4	9.7%	76.0%	14.2%	14.2%	7.9%	4

Table 8 carries a reliability result that the accuracy column alone would hide. The governed condition is the most likely of the four to return something rather than decline: it refused or errored on 14.2% of scoreable attempts against 33.7% for C1 and 38.2% for C3. It converted almost all of that willingness into wrong answers, at a 77.2% wrong rate against 56.2% for C1. For an analyst, a system that answers more often and is wrong more often is a different product risk from one that declines more often, and the two are not interchangeable at equal accuracy. The frozen generation contract does not distinguish a content refusal from a statement that the available schema was insufficient, so those two rates are reported as unavailable rather than inferred.

9.3 Exploratory contrasts

Read as a ladder, Table 9 says one clear thing and one careful thing.

The clear thing is that access to business knowledge is what moved the result. Giving the same agent, the same harness, and the same retrieval budget a searchable copy of the raw knowledge base is worth +12.0 points under the official scorer and +9.4 under the corrected one, on 37 questions gained against 5 lost. That contrast is the one rung where model, scaffold, dialect, and execution contract are all held fixed, so it carries the most weight of any comparison in the study.

The careful thing is that the two rungs after it did not keep the gain. C3 minus C2 is −13.5 points, on 4 questions gained against 40 lost: converting the same knowledge into the bounded structured model and exposing it through the same search interface lost most of what the prose had provided. C4 minus C3 is indistinguishable from zero. Neither of those is a verdict on semantic layers in general, because both depend on how much of this knowledge base the compiler could represent, and Table 4 shows that it could represent 17.7% of it. The honest statement of the negative result is narrow and worth stating precisely: a conservatively compiled semantic model,

Table 9: Prespecified exploratory rung-to-rung contrasts. Intervals are deterministic 95% held-out-item-clustered percentile bootstrap intervals with 10,000 replicates. Gains and losses count questions whose paired outcome changed in each direction.

Scorer	Contrast	Difference	95% interval	Gains	Losses
Official Soft EX	C2 minus C1	12.0%	5.6–18.7%	37	5
Official Soft EX	C3 minus C2	−13.5%	−20.6–−7.1%	4	40
Official Soft EX	C4 minus C1	−1.5%	−7.1–4.1%	10	14
Official Soft EX	C4 minus C3	0.0%	−4.9–4.9%	10	10
Corrected multiset	C2 minus C1	9.4%	3.4–15.7%	30	5
Corrected multiset	C3 minus C2	−10.9%	−17.6–−4.9%	4	33
Corrected multiset	C4 minus C1	−0.4%	−6.4–5.6%	13	14
Corrected multiset	C4 minus C3	1.1%	−4.1–6.7%	13	10

built from these public inputs and deployed without joins or measures, did not reproduce the benefit that the same knowledge gave as searchable prose.

9.4 Efficiency and operational reliability

The same 1,068 immutable attempts preserve provider-reported token usage where available, end-to-end latency, tool activity, and terminal state. These are workload measurements of four complete systems, not estimates of the semantic layer’s isolated cost. Dollar cost is available for the direct provider surface and unavailable for the governed product’s raw job endpoint. Unavailable does not mean zero, and it is recorded as null with a declared reason.

Table 10: Workload and terminal-state measurements over 267 attempts per condition. Latency and token columns give the median with the Tukey interquartile range.

Cond.	Answered	Latency, s	Tokens	Tools	Cost/attempt	Total
C1	67.0%	33.4 (19.6–48.5)	147.6k (63.9–288.2k)	3	\$1.43	\$381.28
C2	74.2%	44.0 (27.6–60.4)	256.8k (137.8–371.2k)	4	\$1.90	\$506.66
C3	63.3%	39.2 (23.5–68.6)	195.3k (89.1–429.4k)	3	\$1.84	\$491.70
C4	85.8%	51.1 (39.8–91.6)	580.8k (410.1–1029.3k)	7	n/a	n/a

The governed condition spent 3.9 times C1’s median tokens, 1.5 times its median latency, and 2.3 times its median tool calls, and did not convert any of it into accuracy. Almost all of the difference is input volume: median input and output tokens were 145.8k and 2.0k for C1 against 580.0k and 3.0k for C4. Because scaffold, agent, and execution path all differ between those two arms, the overhead cannot be attributed to semantic enforcement, but the workload difference itself is a measured product fact.

Failures cost more than answers in every condition. Median observed total tokens for answered against errored attempts were 134k versus 335k in C1, 249k versus 367k in C2, 195k versus 403k in C3, and 574k versus 815k in C4, whose error-token median uses the 36 of 38 errors that carried token telemetry. Governed error latency was 87.5 seconds against 50.5 seconds for answered attempts. The product opportunity there is not only fewer failures but earlier ones: a terminal contract failure or an unsupported result type discovered at the end of an expensive execution path is the worst

place to discover it. Model version, retry count, and validation-attempt count are unavailable for all 267 governed attempts.

9.5 What this result licenses

Three statements are supported. Searchable business knowledge improved an analytical agent’s accuracy on these questions by a margin that survives both scorers and both directions of the bootstrap interval. The governed semantic-layer product, deployed from these public inputs, did not improve on the raw-schema baseline and cost substantially more per attempt. And the governed path failed differently rather than less: it declined far less often and was wrong far more often.

Three statements are not supported, and the study should not be cited for them. This is not a measurement of semantic-layer query composition, because no governed query was composed by the semantic layer (Section 5). It is not a general verdict on Omni, because the deployed model was built by a conservative public-only compiler that published no joins and no measures, and a model authored the way the product’s documentation prescribes is a different system. And it is not a full-benchmark result, because the executed frame covers 89 of the original 101 held-out questions on 16 of the 18 databases.

The second of those unsupported statements is the one a registered successor condition is built to address. Section 10 defines C5, a governed condition deployed the way the product’s documentation prescribes rather than the way a conservative compiler produces, fixes its comparisons in advance, and carries the empty tables its result will fill. It is a development-partition condition and reports nothing here.

10 Registered successor condition C5, and its pending readout

Section 9 reports a governed condition that never composed a query through the semantic layer, and Section 8 explains why: a compiler that declined to guess published a model with no joins and no measures, so no compiled path existed for the planner to take. That leaves the motivating question open rather than answered. C5 is the registered condition that closes it. It was registered on 2026-08-30, after the held-out aggregates were opened; its design provenance is the mechanism measurements and the product’s public documentation, not any held-out outcome. That ordering is why it is confined to the development partition and can never become a held-out candidate. It reports no number in this paper. The placeholders below mark exactly where its numbers will go, so that a later report fills them rather than restating the design.

10.1 What C5 changes

C4 measured the product under a deployment shape its documentation never recommends. C5 measures it under the shape the documentation prescribes, built by a separate compiler entry point from the same public schema, public column meanings, and public HKB, and nothing else. Six changes define the condition:

- A view for every public table, 47 to 63 per database, against the mechanical baseline’s 6 to 11, with a physical dimension for every column carrying its public column meaning as description.
- A join for every foreign-key contract that passes the same conservative cardinality rule the relationship experiment used, 1,049 of 1,228 public foreign keys. The endpoint filters that bounded that experiment relax on their own, because both endpoints are now published.

- The complete HKB carried into `ai_context` rather than pruned: one exact field target becomes field-level context, a table-scoped or cross-table definition becomes topic-level context with its dependency chain inlined prerequisite-first, and database-global knowledge becomes model-level context. Formulas are carried verbatim. The compiler enforces the product’s documented budget, warning at 150,000 characters of model context per database and failing above 175,000.
- Field metadata only through channels the deployment API’s exact-readback contract confirms, which excludes `synonyms` and `sample_values`.
- Generic query-pattern guidance at model level, describing the structurally fragile shapes from the failure taxonomy once each, instantiated from schema structure alone and from no question content.
- No measures. Measures with grain resolution are registered as a second phase and built only if the first phase warrants the spend.

Everything that defends the number is unchanged: public-only content provenance, both frozen scorers reported without post-result selection, one generation, append-only records, isolated `livesqlbench-*` branches, per-attempt exact readback of every deployed document including the model-level context, and the rule that a wrong answer is never a reason to rerun.

10.2 Prespecified comparisons

Table 11 fixes what C5 will be compared against, and on which questions, before any C5 outcome exists. The matched development frame is constructed from public inputs alone, by narrowing each frozen selection to the intersection of the governed execution frame and the direct-baseline freeze, so that one scorer run per selection produces C1 through C5 aggregates over identical questions. That intersection is 122 questions across 14 databases; the matched governed baseline over it holds 122 attempts, 77 answered and 45 errored, against 154 scheduled attempts over 18 databases.

Table 11: Prespecified C5 comparisons. No result exists for any row at the time of writing.

Comparison	Frame	Question it answers	Result
C5 against C2	Matched 122 questions, 14 databases	How much of the +12.0-point knowledge benefit a governed deployment delivers when the model carries the same knowledge	[PENDING]
C5 against frozen C4	136 answerable development questions	Effect of the documentation-idiomatic deployment over the mechanical baseline	[PENDING]
C5 against E02	117 captured questions	Incremental effect beyond declared relationships alone	[PENDING]
C5 query paths	All captured C5 queries	Whether declared joins move governed queries off the product’s rewrite path	[PENDING]

10.3 Deployment state at the time of writing

The arm is in deployment and has not yet generated. Its failures are preserved with the same discipline as the rest of this record, because each one is a property of writing a semantic model to

this product rather than an incident to be tidied away.

The first pass failed on 21 views whose CamelCase table names the product renames on creation. The second reached 8 of 16 databases before the process that owned it exited, leaving a half-deployed arm; the fix ties the remote identity to the deployment run’s revision, so a retry can never land on a populated branch, and long jobs now run detached. The third verified 15 of 16 databases with zero validation issues and exact readback, and failed on `planets_data_large` at the readback stage after 7 observations. A read-only comparison of that branch against the committed bundle showed 58 of its 118 documents differing, every view and no topic, carrying the product’s own schema-generated view content rather than ours: the product regenerated the view layer from the physical schema after the upload landed, on the database with the largest view surface in the set. The fix uploads a second time when a branch does not converge and fails only if the reuploaded branch still differs. The fourth pass carried that fix and reproduced the same single-database readback failure, which makes the behavior deterministic rather than a timing race. A fifth pass is running as this paper is written.

None of these fixes contains a database name, a question, or a label, and none can mask a real mismatch, because a branch that still differs after the second upload fails exactly as before.

10.4 Where the C5 result will go

Tables 12 and 13 are the destinations. The accuracy table is empty in every cell, including the comparator rows, because the matched-frame scorer run has not been executed for any condition; running it for C5 alone would not produce a comparison. The mechanism table carries the frozen governed baseline’s measured values beside empty C5 cells, because those are the numbers C5 is built to move.

Table 12: Mean one-shot execution accuracy on the matched 122-question development frame, both frozen scorers, to be filled by the C5 scorer run.

Condition	Official-compatible	Soft EX	Corrected multiset
C1 raw schema, direct SQL		[PENDING]	[PENDING]
C2 searchable raw HKB		[PENDING]	[PENDING]
C3 searchable exported model		[PENDING]	[PENDING]
C4 mechanical governed baseline		[PENDING]	[PENDING]
C5 documentation-idiomatic governed		[PENDING]	[PENDING]

Table 13: Query-path mechanism readout. The C4 column is measured over the 135 captured governed development queries; the C5 column is to be filled from the same analyzers over C5’s captured queries.

Measurement	Frozen C4	C5
Queries carrying <code>rewriteSql: true</code>	135 of 135	[PENDING]
Queries declaring a join path	0 of 135	[PENDING]
Attempts referencing at least one compiled dimension	109 of 135	[PENDING]
Attempts expanding an HKB-backed derived definition	39 of 135	[PENDING]
Attempts selecting no compiled model field	97 of 135	[PENDING]

Placeholder. *C5 findings. One paragraph stating the accuracy outcome on the matched frame under both scorers, the paired contrast against C2 with its clustered bootstrap interval, and the direction and size of the change against frozen C4, written after the scorer run and before any interpretation.*

Placeholder. *C5 mechanism reading. One paragraph stating whether declared joins moved governed queries off the rewrite path, what share of C5 queries the semantic layer composed, and whether any residual rewrite is explained by the absence of measures in this phase.*

Placeholder. *C5 interpretation. One paragraph relating the outcome to the three statements in Section 9, and, if the mechanism moved, restating which of them the C5 evidence narrows or widens. If the arm fails to deploy or to generate, the disposition and its stopping rule go here instead of a result.*

10.5 What C5 will not establish

Four limits are fixed in advance. C5 is a development-partition condition, and the held-out frame is closed, so no held-out claim will be made for it whatever it shows. It postdates visible held-out aggregates, which is why it may not be promoted into a held-out candidate. It runs a single generation, so a small difference against a comparator will not be separable from repetition noise the way the three-repetition held-out arm allows. And phase 1 publishes no measures, so a null result would not distinguish declared joins failing to help from a model still missing the aggregation contracts that Section 11 identifies as the deeper gap.

11 What a semantic layer has to represent before it can help

The negative governed result is more useful as a specification than as a verdict. Each item below names a capability whose absence is visible in the preserved artifacts, and each is a general property of importing business knowledge into a governed model rather than an observation about one benchmark.

Grain, identity, cardinality, and aggregation need to be first-class, and their absence needs to be visible before deployment. Table 4 is the evidence. A conservative compiler that refuses to guess turned 17.7% of definitions into executable objects and deferred 46.9% of them across an unresolved grain, and the three most common recorded reasons were unknown cardinality, unspecified aggregation, and missing cross-grain identity. Those are the four contracts an import path currently has no way to express. The practical consequence is that a model author cannot tell, before deploying, whether the model carries enough structure to govern anything. A dry run that reports which definitions compiled, which were kept as context, which were deferred, and why, converts a silent coverage problem into a decision. Without it, the choice is between silently guessed semantics and a large volume of prose whose discoverability by the agent is unknown.

Relationship coverage is the same problem one level up. The public schemas of these 16 databases contain 1,049 foreign keys that pass a conservative cardinality contract, and the bounded modeled candidate could expose 91 relationships. A model author needs to see that gap, with accepted, deferred, and unreachable relationships enumerated, before deciding whether the model has enough structure for a governed query path to exist at all. In this study the gap was not visible until the query-path measurement showed that no governed query had ever declared a join.

Retrieval payloads belong in the agent contract. The first direct-SQL attempt in this study returned all 51 tables of a database and then exhausted its budget before writing a line of SQL (Table 5). A semantically valid tool call ended the run. Schema and semantic search should be query-directed, bounded, and observable, with per-call context volume in telemetry; and typed public provenance identifiers should exist so that a safety filter does not reject canonical foreign-key values as suspected secrets, which is the second failure the same slice exposed.

Not answering has several causes, and the product currently reports one. A content refusal, an explicit statement that the available schema is insufficient, an exhausted model budget, and an infrastructure failure imply four different product actions. The governed condition reports them as one terminal state, which is why its refusal rate is unavailable in Table 10 rather than estimated. Governed jobs should expose a typed refusal outcome, and both the product and comparator telemetry should retain the raw state even when a report also groups them into broader reliability categories.

The result contract over rewritten SQL is the one nobody wrote down. Thirty-one of the 34 governed non-answers in the development baseline shared an UNKNOWN selected-field type, and the relationship experiment then preserved 14 generated semantic queries that could not be captured because their result types were unsupported. That is not a scoring inconvenience; it is a gap in the interface. When the agent authors the query, neither side currently specifies what the planner guarantees about the type of an output column that the semantic model never declared. Production planning and execution need a stable, total representation for unknown, boolean, temporal, and null values, and an unsupported planner type should surface as a visible product outcome rather than an adapter exception that obscures whether the governed query itself succeeded. The same argument covers truncation: a preview is not an analytical result, and presentation-control records do not belong in data rows.

Ordering. These are not independent. Result typing comes first, because until an unsupported output type stops silently consuming a governed attempt, no downstream experiment can be scored cleanly, as E02's 14 uncaptured queries demonstrate. Grain and relationship authoring comes next, because it is what would give a planner something to compile. Retrieval bounds, refusal typing, and deployment diagnostics are independent of both and cheaper than either.

The registered C5 condition tests the second of these directly. It publishes the joins and the full knowledge context that the mechanical baseline lacked, so its query-path readout in Table 13 measures whether relationship coverage alone is enough to move a governed agent off the rewrite path, or whether the grain and aggregation contracts have to arrive with it. Its deployment history already exercises two of the cheaper items: the readback divergence in Section 10 is the canonical-export gap, and the branch that had to be uploaded twice is the deployment-identity gap.

12 Threats to validity and limitations

Every item in this section except the last three was written down before any result existed, which is why none of them can be introduced now as an explanation for an inconvenient outcome. The last three are deviations the executed study incurred, and they are stated at the same level of detail as the preregistered ones.

- **Scope.** Because all 18 databases appear in both partitions, the headline estimates unseen-question performance inside a modeled database, not cold-start performance on an unmodeled one.
- **Supervision.** The protocol permitted offline use of training failures and training gold. The executed final candidate used neither, and the metered partition was never consumed, so the reported system is the frozen mechanical baseline rather than a tuned one.
- **Contamination.** The benchmark is public. The maintainers’ design claim cannot rule out post-release model exposure, and this project cannot inspect training provenance. Paired within-question contrasts are less vulnerable than absolute levels, which is one reason the C2 minus C1 result carries more weight here than any single accuracy figure.
- **Power.** 89 held-out questions give roughly one percentage point of resolution per question. The knowledge contrast is large enough to clear that, at +12.0 points with a 5.6-point lower bound. The governed contrasts are not: C4 minus C1 at −1.5 points has an interval of −7.1 to 4.1, so this study cannot distinguish a small governed loss from a small governed gain.
- **Metric.** Soft EX deliberately equates some semantically different SQL and rejects some extensionally equal results. Accuracy is accuracy under a pinned scorer, not proof of semantic equivalence, which is why the second comparator is reported alongside it everywhere.
- **Model parity.** The governed condition’s underlying models could not be matched or observed, so C4 minus C3 is a system-level comparison and not a causal estimate of enforcement.
- **Query path.** This limitation is separate from model parity and is not removed by achieving it. The governed condition never exercised semantic query composition: every governed query took the product’s raw-SQL rewrite path, and the deployed model declared no join path and no measure for a planner to compile. C4 minus C3 therefore differs on the agent, on field resolution at rewrite time, on the accessible surface, and on the execution contract, and not on who composes the query.
- **Governed structural aggregates.** Because `generated_sql` is null on every governed attempt, the governed arm’s relation and join counts are computed from the SQL text carried on each semantic query. They describe agent-written queries in all four conditions and support no claim about compiled query structure.
- **Harness.** Published controlled studies place the scaffold effect anywhere from 0 to 28 percentage points [33, 32], so all condition results are conditional on the disclosed harnesses, and effects may appear in error type, cost, retries, or reliability rather than in aggregate accuracy. The reliability and workload differences in Section 9 are an instance of exactly that.
- **Comparator scaffold choice.** The four-table schema window is a comparator design decision supported by one question’s evidence. The 20-question sensitivity arm that would test it was frozen and then deferred before execution.
- **Stochasticity.** Three repetitions characterize some instability but do not fully identify provider or temporal variance.
- **Environment parity.** Any mismatch between the database the evaluated product queries and

the scorer’s snapshot can manufacture false gains or false failures, which is why fingerprint and canary parity checks precede every run.

- **Version drift.** The product, the model providers, and the evolving benchmark can all change during an evaluation of this size. Pinned artifacts, timestamps, deterministic interleaving, and the sealed generate-then-score boundary mitigate this without eliminating it.
- **Scalability of the modeling claim.** Manual interpretive mappings and benchmark-specific exceptions weaken any claim that semantic-model construction is automatic, and are counted, not described qualitatively.
- **Composite systems.** A production system that routes across multiple opaque model tiers and includes a validation stage cannot be reduced to one model label, and is not reported as though it could.
- **Executed frame (deviation).** The held-out frame covers 89 of the original 101 sealed questions on 16 of the 18 databases. The 12 excluded questions sit on the two databases the official Linux loader cannot populate, and the exclusion was made before any sealed generation, label release, or outcome access, using only loader behavior. It narrows the estimand; it does not create missing outcomes. The same defect makes 18 development questions unscorable in the official environment. Loading the omitted archive files would restore those questions at the cost of comparability with the official environment, which is why it was not done.
- **Development-baseline scope (deviation).** The direct-SQL development baseline excludes two further databases and therefore describes those conditions on 16 databases. Any comparison against a wider governed frame must use matched coverage or disclose the mismatch; the alignment in Section 8 does the former.
- **Scoring order (deviation).** The untuned held-out arm was scored before the E02 development experiment finished, reversing the intended order. The frozen untuned comparison is unaffected, because every held-out generation completed before scoring and both scorers were published together. The consequence is that no held-out claim about optimization can be made at all: E02 resolves inconclusive under its own coverage rule, and the visible held-out aggregates may not drive any further intervention, checkpoint, or promotion.

13 Standing of the record, and what is still open

What is closed. The four-condition held-out comparison is complete and terminal. All 1,068 generations executed under Freeze B, both frozen scorers ran in one atomic pass, and the aggregates in Section 9 are the result. Nothing in this study may reopen them: the no-rerun rule forbids regenerating a trial because its answer was wrong, the metered checkpoint partition is unconsumed and stays that way, and the visible held-out aggregates may not drive a further intervention or a promotion. The development record is likewise closed, with E01, E02, and E05 resolved inconclusive by their own stopping rules and E03 and E04 never run.

What is still under human control. Per-question held-out labels remain in custody. Only identity-free aggregates have been released, which is what allows this paper to report accuracy without reporting which questions were answered. The prespecified post-hoc analyses that need those labels, the gold-SQL template-overlap audit and the blinded gold-quality adjudication, therefore remain specified and unexecuted, behind a human-controlled release. Their absence is a gap in the analysis, not an ambiguity in the result: neither could change the primary endpoints, both were declared as separate sensitivity analyses, and the original official score stands regardless.

What the failure taxonomy now contains. The protocol version of this paper listed three failure hypotheses and marked every taxonomy cell unmeasured. Two of the three are now measured, and both were partly right for the wrong reason. HKB dependency composition was predicted to lose meaning during transformation; what the record shows is a compiler that declined to represent 46.9% of definitions rather than one that represented them badly, and the loss codes name grain, cardinality, and aggregation as the missing contracts. Semantic discoverability was predicted to hide correct definitions from the retrieval surface; what the record shows is that the product did reach compiled dimensions, on 109 of 135 governed development attempts, and then wrote its own SQL around them. The third hypothesis, compilation and validation fidelity, is untested in the sense it was posed, because no semantic compilation occurred on any governed attempt.

What is running. One successor condition is in deployment and reports nothing in this paper. Section 10 defines C5 in full: a governed condition built the way the product’s own documentation prescribes, from the same public inputs under the same custody rules, confined to the development partition. Its comparisons, its frames, its stopping rules, and the empty tables its numbers will fill are fixed there in advance. Four deployment passes are preserved with their failures, the most recent verifying 15 of 16 databases and failing exact readback on the one whose view layer the product regenerates after upload; a fifth is running as this paper is written.

The reason to register it in this record rather than to leave it for a later one is that it addresses a limitation this study cannot repair. The held-out frame is closed, C5 postdates visible held-out aggregates, and no held-out claim will be made for it. What it can settle is whether the governed rung fails as a design or failed here as a deployment, which is the question Section 9 leaves open and the one a reader of this paper is most likely to ask next.

14 Data and materials availability

The public dataset is LiveSQLBench Large-v1 at revision `a418e108d5cbb4cf9b783a928eff5e924ad2460d` (<https://huggingface.co/datasets/birdsql/livesqlbench-large-v1>), and the official harness and evaluator are read at commit `e15cd221267e06fabfaf6a3d4a69308280ce9a7c` (<https://github.com/bird-bench/livesqlbench>). The dataset card carries an internal license inconsistency, CC-BY-4.0 in its front matter and CC-BY-SA-4.0 in its footer, which is recorded here because it constrains redistribution of derived public artifacts.

The preregistration artifacts required to reproduce the population and both splits are the committed eligible manifest, train and test identifier lists, manifest and split metadata, and the development split metadata, together with the preparation and split scripts and the pinned seeds `omni-livesqlbench-large-v1-split-v1` and `omni-livesqlbench-large-v1-development-split-v1`. Their content hashes are given in Section 4, and regeneration from the pinned source hash is verified by test rather than asserted.

Private benchmark gold, hidden knowledge annotations, hidden test cases, run artifacts, raw traces, and credentials are not redistributed and are excluded from version control by construction: the custody tools never write held-out records into the workspace, artifact validators reject protected fields recursively at any nesting depth, and raw traces and generated SQL are confined to ignored run roots. Aggregate results, hashes, counts, and permitted summaries are the only correctness-adjacent material that appears in committed artifacts.

The executed result adds four classes of artifact, all identity-free. The generation and score records for every attempt are hash-bound to each other and to the Freeze-B manifest, with the

commits and digests given in Section 4. The deterministic telemetry aggregate behind Table 10 hashes to 7a614d6c861d4d2a982501ea8c89b2817820d965671caf800cc155759be481a8, and the official aggregate receipt to 0296753e8fcfb826a99ed2f86088ecd6b61981db8dea47d93e7871cef2690a78. The frozen governed baseline’s immutable selection hashes to 256145c13cfae7142d92f108b4ee9dd93e658a44cafb683e5aec90170b8315cc, and an append-only recovery manifest accounting for all 45 original capture failures, 11 result-only replays and 34 explicit evaluated-system failures, hashes to 5d6ff474f30d3de6d703ad5c6c59373fe8093515eabb83473bdb352c4f30fd9f. The E02 candidate set hashes to 12c4e1a8cab38f0f47e14b5c553c87c800ca07f27bae568171f1d7caaf7589a7, and its no-rerun diagnostic is published as a separate aggregate artifact. The repository’s evidence index maps each claim in this paper to the artifact that carries it.

The primary results report, the product-findings ledger, the failure taxonomy, the governed query-path disclosure, and the contemporaneous research log all live in the same public repository as this source. The query-path disclosure is the one to read alongside Section 5: it lists every earlier statement in the project that the measurement retracted, including the condition table that described the governed arm as enforcing semantic compilation.

The comparison that motivated this ablation is the published benchmark of Omni’s agentic analytics harness (<https://omni.co/blog/benchmarking-omnis-agentic-analytics-harness>). This study tested that claim under a design that separates knowledge access, representation, and enforcement. It did not attempt to reproduce it.

References

- [1] Robert E. Blackwell; Jon Barry; Anthony G. Cohn (2024). Towards Reproducible LLM Evaluation: Quantifying Uncertainty in LLM Benchmark Scores. arXiv:2410.03492. <https://arxiv.org/abs/2410.03492>
- [2] Ruisheng Cao; Fangyu Lei; Haoyuan Wu; Jixuan Chen; Yeqiao Fu; Hongcheng Gao; Xinzhuang Xiong; Hanchong Zhang, et al. (2024). Spider2-V: How Far Are Multimodal Agents From Automating Data Science and Engineering Workflows?. arXiv:2407.10956. <https://arxiv.org/abs/2407.10956>
- [3] Peter Baile Chen; Devin Yang; Weiyue Li; Fabian Wenz; Yi Zhang; Nesime Tatbul; Michael Cafarella; Çağatay Demiralp, et al. (2024). BEAVER: An Enterprise Benchmark for Text-to-SQL. arXiv:2409.02038. <https://arxiv.org/abs/2409.02038>
- [4] Junming Chen; Junyang Jiang; Xu Chen; Zibo Liang; Kai Zheng (2026). DBA-Bench: A Production-Fidelity Benchmark for LLM-Based Database Operations Agents. arXiv:2607.22165. <https://arxiv.org/abs/2607.22165>
- [5] Guillaume Coqueret; Joan Llull; Florian Oswald; Christophe Pérignon; Christoph Scheuch; Lars Vilhuber (2026). Randomness in large language models: What researchers need to know (and report). arXiv:2607.24372. <https://arxiv.org/abs/2607.24372>
- [6] Cynthia Dwork; Vitaly Feldman; Moritz Hardt; Toniann Pitassi; Omer Reingold; Aaron Roth (2015). Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629. <https://arxiv.org/abs/1506.02629>
- [7] Vitaly Feldman; Roy Frostig; Moritz Hardt (2019). The advantages of multiple classes for reducing overfitting from test set reuse. arXiv:1905.10360. <https://arxiv.org/abs/1905.10360>

- [8] Maria I. Gorinova; Macey Baker; Amy Heineike; Maksim Shaposhnikov; Rob Willoughby; Dru Knox (2026). Position: Coding Benchmarks Are Misaligned with Agentic Software Engineering. arXiv:2606.17799. <https://arxiv.org/abs/2606.17799>
- [9] Mohammadamin Habibollah; Davood Rafiei (2026). SynSQL: Synthesizing Relational Databases for Robust Evaluation of Text-to-SQL Systems. arXiv:2604.27261. <https://arxiv.org/abs/2604.27261>
- [10] Moritz Hardt (2017). Climbing a shaky ladder: Better adaptive risk estimation. arXiv:1706.02733. <https://arxiv.org/abs/1706.02733>
- [11] Rocky Klopfenstein; Yang He; Andrew Tremante; Yuepeng Wang; Nina Narodytska; Haoze Wu (2025). SpotIt: Evaluating Text-to-SQL Evaluation with Formal Verification. arXiv:2510.26840. <https://arxiv.org/abs/2510.26840>
- [12] Morris Lee (2026). Business Truth, not SQL Accuracy: A Rule-Gated 7B Analytics Agent Outperforms a Direct-Prompted 32B Baseline. arXiv:2608.09254. <https://arxiv.org/abs/2608.09254>
- [13] Gavin Leech; Juan J. Vazquez; Niclas Kupper; Misha Yagudin; Laurence Aitchison (2024). Questionable practices in machine learning. arXiv:2407.12220. <https://arxiv.org/abs/2407.12220>
- [14] Fangyu Lei; Jixuan Chen; Yuxiao Ye; Ruisheng Cao; Dongchan Shin; Hongjin Su; Zhaoqing Suo; Hongcheng Gao, et al. (2024). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763. <https://arxiv.org/abs/2411.07763>
- [15] Jinyang Li; Binyuan Hui; Ge Qu; Jiayi Yang; Binhua Li; Bowen Li; Bailin Wang; Bowen Qin, et al. (2023). Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. arXiv:2305.03111. <https://arxiv.org/abs/2305.03111>
- [16] Yucheng Li; Frank Guerin; Chenghua Lin (2023). An Open Source Data Contamination Report for Large Language Models. arXiv:2310.17589. <https://arxiv.org/abs/2310.17589>
- [17] Yue Li; Ran Tao; Derek Hommel; Yusuf Denizay Dönder; Sungyong Chang; David Mimno; Unso Eun Seo Jo (2025). Agent Bain vs. Agent McKinsey: A New Text-to-SQL Benchmark for the Business Domain. arXiv:2510.07309. <https://arxiv.org/abs/2510.07309>
- [18] Chengxi Liao; Tao Xu; Zulong Chen; Chuanfei Xu; Yiyang Wang; Xinyun Wang; Yanlong Zhang; Xiaojun Chen, et al. (2026). EntSQL: A Benchmark for Grounding Text-to-SQL in Long-Context Enterprise Knowledge. arXiv:2606.03363. <https://arxiv.org/abs/2606.03363>
- [19] Xinyu Liu; Shuyu Shen; Boyan Li; Nan Tang; Yuyu Luo (2025). NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation. arXiv:2503.11984. <https://arxiv.org/abs/2503.11984>
- [20] Lovish Madaan; Aaditya K. Singh; Rylan Schaeffer; Andrew Poulton; Sanmi Koyejo; Pontus Stenetorp; Sharan Narang; Dieuwke Hupkes (2024). Quantifying Variance in Evaluation Benchmarks. arXiv:2406.10229. <https://arxiv.org/abs/2406.10229>
- [21] Evan Miller (2024). Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations. arXiv:2411.00640. <https://arxiv.org/abs/2411.00640>

- [22] Rahmatullah Musawi; Sheng Lu (2025). Towards Contamination Resistant Benchmarks. arXiv:2505.08389. <https://arxiv.org/abs/2505.08389>
- [23] Elias Chaibub Neto; Bruce R Hoff; Chris Bare; Brian M Bot; Thomas Yu; Lara Magravite; Andrew D Trister; Thea Norman, et al. (2016). Reducing overfitting in challenge-based competitions. arXiv:1607.00091. <https://arxiv.org/abs/1607.00091>
- [24] Shiwen Ni; Xiangtao Kong; Chengming Li; Xiping Hu; Ruifeng Xu; Jia Zhu; Min Yang (2024). Training on the Benchmark Is Not All You Need. arXiv:2409.01790. <https://arxiv.org/abs/2409.01790>
- [25] Yonatan Oren; Nicole Meister; Niladri Chatterji; Faisal Ladhak; Tatsunori B. Hashimoto (2023). Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623. <https://arxiv.org/abs/2310.17623>
- [26] Wenqi Pei; Shizheng Hou; Boyan Li; Han Chen; Zhichao Shi; Yuyu Luo (2026). ROSE: An Intent-Centered Evaluation Metric for NL2SQL. arXiv:2604.12988. <https://arxiv.org/abs/2604.12988>
- [27] Mohammadreza Pourreza; Davood Rafiei (2023). Evaluating Cross-Domain Text-to-SQL Models and Benchmarks. arXiv:2310.18538. <https://arxiv.org/abs/2310.18538>
- [28] Divyanshu Saxena; Rishikesh Maurya; Xiaoxuan Ou; Gagan Somashekar; Shachee Mishra Gupta; Arun Iyer; Yu Kang; Chetan Bansal, et al. (2025). Continuous Benchmark Generation for Evaluating Enterprise-scale LLM Agents. arXiv:2511.10049. <https://arxiv.org/abs/2511.10049>
- [29] Juan Sequeda; Dean Allemang; Bryon Jacob (2023). A Benchmark to Understand the Role of Knowledge Graphs on Large Language Model’s Accuracy for Question Answering on Enterprise SQL Databases. arXiv:2311.07509. <https://arxiv.org/abs/2311.07509>
- [30] Shannon Serrao; Soumitra Chatterjee; Dorina Strori; Abhishek Sharma; Nathan Miller (2026). BADGER: Bridging Agentic and Deterministic Evaluation for Generative Enterprise Reasoning. arXiv:2606.02109. <https://arxiv.org/abs/2606.02109>
- [31] Weijia Shi; Anirudh Ajith; Mengzhou Xia; Yangsibo Huang; Daogao Liu; Terra Blevins; Danqi Chen; Luke Zettlemoyer (2023). Detecting Pretraining Data from Large Language Models. arXiv:2310.16789. <https://arxiv.org/abs/2310.16789>
- [32] Jason Starace (2026). Scaffold Effects on GAIA: A Controlled Comparison. arXiv:2606.08529. <https://arxiv.org/abs/2606.08529>
- [33] Naman Vats; Oleg Golev (2026). The Scaffold Effect in Coding Agents: Harness Choice as a Hidden Variable in Coding-Agent Evaluation. arXiv:2607.22585. <https://arxiv.org/abs/2607.22585>
- [34] Anoushka Vyas; Aarushi Dhanuka; Sina Khoshfetrat Pakazad; Henrik Ohlsson (2026). Data Intelligence Agents: Interpreting, Modeling, and Querying Enterprise Data via Autonomous Coding Agents. arXiv:2606.19319. <https://arxiv.org/abs/2606.19319>
- [35] Colin White; Samuel Dooley; Manley Roberts; Arka Pal; Ben Feuer; Siddhartha Jain; Ravid Shwartz-Ziv; Neel Jain, et al. (2024). LiveBench: A Challenging, Contamination-Limited LLM Benchmark. arXiv:2406.19314. <https://arxiv.org/abs/2406.19314>

- [36] Ruijie Xu; Zengzhi Wang; Run-Ze Fan; Pengfei Liu (2024). Benchmarking Benchmark Leakage in Large Language Models. arXiv:2404.18824. <https://arxiv.org/abs/2404.18824>
- [37] Tao Yu; Rui Zhang; Kai Yang; Michihiro Yasunaga; Dongxu Wang; Zifan Li; James Ma; Irene Li, et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. arXiv:1809.08887. <https://arxiv.org/abs/1809.08887>
- [38] Yunbei Zhang; Janet Wang; Yingqiang Ge; Weijie Xu; Jihun Hamm; Chandan K. Reddy (2026). Stop Comparing LLM Agents Without Disclosing the Harness. arXiv:2605.23950. <https://arxiv.org/abs/2605.23950>
- [39] Fuheng Zhao; Jiayue Chen; Lawrence Lim; Ishtiyaque Ahmad; Divyakant Agrawal; Amr El Abbadi (2023). LLM-SQL-Solver: Can LLMs Determine SQL Equivalence?. arXiv:2312.10321. <https://arxiv.org/abs/2312.10321>